A

MAJOR PROJECT REPORT

ON

Certificate Verification System

Submitted in partial fulfillment of the requirement

for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

INFORMATION TECHNOLOGY

BY

B.ABHISHEK(21P61A1216)

G.RITHVIK GOUD(21P61A1248)

K. VARSHITH REDDY(21P61A1262)

D. SOHAN (21P61A1232)

Under the esteemed guidance of

Mrs. B. Manjulatha

Assistant Professor

Department of IT

Department of Information Technology

VIGNANA BHARATHI INSTITUTE OF TECHNOLOGY

(Approved by AICTE, Accredited by NBA, NAAC, Permanently Affiliated to JNTUH)

Aushapur (v), Ghatkesar (m), Medchal (d), TELANGANA-501 301

Academic Year 2024-25

VIGNANA BHARATHI Institute of Technology

AUTONOMOUS Under UGC

Accredited by NBA NATIONAL BOARD OF ACCREDITATION (ECE,EEE,CSE,IT,CE) NAAC NATIONAL ASSESSMENT AND ACCREDITATION COUNCIL A Grade

AUSHAPUR (V), GHATKESAR (M), MEDCHAL (D)-501 301

Department of Information Technology

CERTIFICATE

This is to certify that the project entitled **"Certificate Verification System"** is being submitted by **B. ABHISHEK (20P61A1216)** in partial fulfillment of the requirement for the award of the degree of Bachelor of Technology in Information Technology is a record of bonafide work carried out by them under my guidance and supervision during the academic year 2024- 2025.

The results embodied in this project report have not been submitted to any other University for the award of any degree.

Internal Guide

Mrs. B. Manjulatha

Assistant Professor

Department of IT

Head of the Department

Dr. K. Kalaivani

Associate Professor

Department of IT

Project Coordinator

Mrs. K. Soumya

Assistant Professor

Department of IT

External Examiner

Name:

College:

AUSHAPUR (V), GHATKESAR (M), MEDCHAL(D)-501301

Department of Information Technology

DECLARATION

I, **B. ABHISHEK** bearing hall ticket number **21P61A1216**, here by declare that the project report entitled "**CERTIFICATE VERIFICATION SYSTEM** " under the guidance of **Mrs. B. Manjulatha** , Department of Information Technology, **VBIT**, Hyderabad, is submitted in partial fulfilment of the requirement for the award of the degree of Bachelor of Technology in Information Technology.

This is a record of bonafide work carried out by us and the results embodied in this project have not been reproduced or copied from any source. The results embodied in this project report have not been submitted to any other university or institute for the award of any other degree.

B. ABHISHEK (21P61A1216)

ACKNOWLEDGEMENT

First and foremost, I wish to express my gratitude towards the institution "**Vignana Bharathi Institute Of Technology**" for fulfilling the most cherished goal of our life to do Bachelor of Technology**.**

It is great pleasure in expressing deep sense of gratitude to my Internal guide, **Mrs. B. Manjulatha** , Assistant Professor, Department of Information Technology for his valuable guidance and the freedom he gave us.

I also express our sincere thanks to **Mrs. K. Soumya**, **Project Coordinator** for her encouragement and support throughout the project.

I am deeply grateful to **Dr. K. Kalaivani**, Head of the Department, Department of Information Technology for granting us the opportunity to conduct this project.

I take immense pleasure in thanking **Prof. P.V.S. Srinivas, Principal, Vignana Bharathi Institute of Technology, Hyderabad** for having permitted us to carry out this project work. Our utmost gratitude also goes to all the **FACULTY MEMBERS** and **NON-TEACHING STAFF** of the Department of Information Technology for their support throughout my project work.

B. ABHISHEK (21P61A1216)

VIGNANA BHARATHI INSTITUTE OF TECHNOLOGY
Department of Information Technology

COURSE OUTCOMES

Course: Major Project Class: IV B. Tech II Semester

AY: 2024-2025

Course Outcomes
After completing the Projects, the student will be able to:

Code	Course Outcomes	Taxonomy
C424.1	Identify and state the problem precisely to prepare the abstract	Remember
C424.2	Analyze the existing system, and outlining the proposed methodology for effective solution	Analyze
C424.3	Use various modern tools for designing applications based on specified requirements	Apply
C424.4	Develop applications with adequate features and evaluate the application to ensure the quality	Create
C424.5	Prepare the document of the project as per the guidelines	Create

PROGRAM OUTCOMES (POs)

PO1 Engineering knowledge: Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.

PO2 Problem analysis: Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.

PO3 Design/development of solutions: Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.

PO4 Conduct investigations of complex problems: Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid

conclusions.

PO5 Modern tool usage: Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.

PO6 The engineer and society: Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.

PO7 Environment and sustainability: Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.

PO8 Ethics: Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.

PO9 Individual and team work: Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

PO10 Communication: Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.

PO11 Project management and finance: Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.

PO12 Life-long learning: Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

PROGRAM SPECIFIC OUTCOMES (PSO's)

PSO 1	Simulate computer hardware and apply software engineering principles and techniques to develop various IT applications
PSO 2	Analyze various networking concepts and also aware of how security policies, standards and practices are used for trouble-shooting.
PSO 3	Design and maintain database for providing back-end support to software projects.
PSO 4	Apply algorithms and programming paradigms to produce IT based solutions for the real-world problems.

VIGNANA BHARATHI INSTITUTE OF TECHNOLOGY
Department of Information Technology

COs Mapping with PO/PSO

Project Title: Certificate Verification System

Name of the Supervisor: Mr. A. Naveen Kumar

Batch Details:

S.NO.	Regd. No.	Student Name	Technology
1	21P61A1216	B ABHISHEK	Web Development

CO-PO Mapping for Major Project:

High -3 Medium –2 Low- 1

PO / CO	PO 1	PO 2	PO 3	PO 4	PO 5	PO6	PO 7	PO8	PO 9	PO 10	PO1 1	PO 12	PS O 1	PSO 2	PSO 3	PSO 4
C424. 1	3	3	2	3	3	2	-	3	3	3	3	3	2	-	3	2
C424. 2	3	3	3	3	3	2	-	3	3	3	3	3	2	-	3	3
C424. 3	3	3	3	3	3	2	-	3	3	3	3	3	2	-	3	3
C424. 4	3	3	3	2	3	3	-	3	3	2	3	3	3	-	3	2
C424. 5	3	3	3	3	3	3	-	3	3	3	3	3	2	-	3	3
AVG	3	3	2. 8	2. 8	3	2.5	-	3	3	2. 8	3	3	2	-	3	2.6

CO-PO Mapping Justification

Course: Major Project Class: IV B. Tech II Semester

AY: 2024-2025

Mapped POs:

PO1	Student attains a basic knowledge and engineering fundamentals to identify and state the problem.
PO2	Students are able to literature survey and analyze complex problems to develop solutions.
PO3	Students are made able to design solutions for complex problem and design software components, process to meet specifications.
PO4	Students are made able to use knowledge and research methods using design of experiments, analytical skills to interpret data and synthesis of information to provide valid conclusions.
PO5	Students are made able to develop web applications by using Integrated Modern Tools using Java
P06	Students are made able to develop web application it helps for society
PO7	Students are made their project environment friendly
P08	Students are made able to develop web application by including ethics while developing application
PO9	Students are made able to work effectively as an individual, member or leader in a team.
PO10	Students are made able to work effectively as they communicate with each other while developing their project
PO11	Students are made able to Apply management principles and apply these to one's own work
PO12	Students are able to engage in independent and life-long learning and possess the knowledge and skills for employability by doing the projects.

Mapped PSOs:

PSO1	Students are made able to apply software engineering principles and techniques to develop the web application for Online Integrated examination system.
PSO2	Students are made able to Analyze various networking concepts and also aware of how security policies, standards and practices are used for trouble shooting
PSO3	Students are made able to design and maintain database by providing back-end support.
PSO4	Students are able to use ML programming paradigms to offer IT-based solutions for Online Integrated examination system.

Supervisor Signature

ABSTRACT

With the rapid digitalization of educational and professional certifications, the authenticity of digital documents has become a growing concern. The rise in certificate fraud—especially the manipulation or duplication of QR-embedded documents—poses a serious threat to institutions, employers, and individuals alike. To tackle this, our project presents an intelligent certificate verification system capable of automatically scanning uploaded PDF files, detecting embedded QR codes, and validating the certificate contents against the information presented visually in the document.

This system utilizes PyMuPDF for precise text and image extraction, and an external QR scanning API for reliable decoding of embedded QR content. The verification logic is tailored to institutions like Infosys, where a JSON response from the QR code is cross-verified with visible PDF text to classify documents as genuine or fake. The system supports batch uploads, processes multiple files concurrently using multithreading for speed, and provides a clear summary of results—highlighting valid and fraudulent certificates.

What sets this solution apart is its ability to extract the actual name displayed on the certificate (not just from QR) and maintain detailed reporting, including original and fake uploaders. With a simple frontend interface and robust backend logic, this system delivers an efficient and scalable approach to digital certificate validation—helping institutions and recruiters confidently verify credentials.

INDEX

LIST OF FIGURES

LIST OF TABLES

CHAPTER–1

INTROD
UCTION

1. INTRODUCTION

1.1 INTRODUCTION TO THE SYSTEM

The Certificate Verification System is a modern, intelligent software solution designed to automate the process of validating digital certificates issued by educational institutions, corporate training programs, and certification bodies. As the world transitions toward paperless operations, digital certificates have become a convenient method for credentialing. However, this convenience comes with increased risk—fake certificates can now be easily created and circulated. To combat this, our system leverages document scanning, QR code decoding, text extraction, and pattern matching techniques to accurately verify the authenticity of submitted PDF certificates. By utilizing a combination of PDF parsing, QR reading APIs, and rule-based validation checks, the system ensures that every certificate undergoes rigorous scrutiny before being classified. This application supports batch uploads and delivers comprehensive feedback by identifying original certificates, flagging duplicates, and mapping out forgery attempts with exact name mismatches. The system architecture combines backend logic built with Django and intelligent OCR/QR APIs, along with a dynamic frontend interface developed using React, providing real-time feedback to users.

1.2 PROBLEM STATEMENT

With the increasing prevalence of online learning platforms and virtual training environments, the issuance of digital certificates has become routine. However, this surge has also led to a parallel increase in certificate forgery, impersonation, and misrepresentation. A growing number of individuals now exploit freely available design tools to alter existing certificates or generate entirely fabricated ones—sometimes by editing only the name, date, or course while retaining an authentic QR code. In such cases, even scanning the QR might lead to genuine verification data, giving the illusion of a valid certificate, when in reality the printed information is misleading.

Manual verification processes used by many institutions today are no longer feasible or reliable when dealing with hundreds or thousands of digital certificates. They are time-intensive, susceptible to human error, and often fail to detect subtle alterations. Moreover, these processes provide no consistent framework for identifying repeated misuse, where the same certificate is modified and reused under different names. The lack of an integrated, intelligent, and auditable solution leads to credibility risks for educational institutions and trust issues for employers or agencies trying to validate a candidate's qualifications.

Thus, the problem we are addressing is not simply whether a certificate exists—but whether the content shown to the viewer actually matches what is encoded in the system of record, and whether it has been tampered with in any form. A certificate that appears valid on the surface may still represent a fraudulent identity if only the QR is relied upon for validation.

1.3 OBJECTIVE

The primary objective of the Certificate Verification System is to provide a fully automated, scalable, and intelligent solution that ensures the authenticity of digital certificates by cross-verifying the embedded QR code data with the visible text displayed in the PDF. Rather than relying on a single point of validation, this system adopts a dual-check strategy: 1) Extracting and decoding the QR code using an external API to retrieve the JSON data (especially the "issued to" field), and 2) Parsing the visible PDF content using PyMuPDF to identify the name printed on the certificate.

One key goal is to identify mismatches between these two sources of information. If the name printed on the certificate differs from the name stored in the QR, the certificate is classified as fake, and the system keeps a detailed log mapping the fake name to the legitimate one. This dictionary-like structure is used for audit trails and accountability. For original certificates, the system ensures that each name is only counted once, preventing duplication and reinforcing the uniqueness of records.

The system is also optimized for batch processing. Users can upload an entire folder of certificates, and the backend will process them concurrently using multi-threading, improving performance while maintaining accuracy. It ensures that valid certificates are moved into a secure "original" directory, while fake or mismatched ones are segregated into a "fake" directory for review. This organized structure not only helps in systematic tracking but also serves as documentation for compliance or reporting.

Furthermore, the system is designed to be integrable and extensible. Its modular nature allows for future enhancements, such as adding watermark detection, blockchain-based verification, or integrating with institutional databases to fetch records dynamically.

1.4 AIM OF THE PROJECT

The ultimate aim of this project is to create a dependable verification system that protects the integrity of digital certificates, prevents academic or credential fraud, and builds trust across all stakeholders—be it issuing authorities, validating organizations, or end users. Through this project, we aim to enable transparent, intelligent, and real-time certificate verification without requiring complex infrastructure or manual scrutiny. Our goal is to provide a platform that not only flags fake certificates but also offers detailed insights into the nature of the forgery. By highlighting discrepancies between QR data and certificate content, the system gives institutions a clear view of misuse patterns, repeat offenders, and even potential risks to their reputation. It's not just about moving a file to a folder—it's about knowing who tried to fake what, why it failed, and who the real owner is.

Additionally, the project is structured to be user-friendly and resource-efficient, ensuring that organizations with minimal technical expertise can deploy it with ease. The seamless integration of Python-based PDF analysis, third-party QR code reading services, and frontend feedback mechanisms ensures a smooth user experience while maintaining a high standard of security and correctness. In essence, this system aspires to become a gold standard for certificate validation in digital-first environments—accurate, traceable, scalable, and above all, trustworthy.

CHAPTER – 2

LITERATURE SURVEY

2. LITERATURE SURVEY

2.1 Survey

2.1.1 Title: "A Blockchain-Based Educational Record Repository and e-Diploma System"

Published By: Pedro V. R. Souza, Claudio J. B. de Oliveira, and Patricia Takako Endo

Published Year: 2023

Summary: This paper presents a novel approach to securely managing educational certificates and diplomas using blockchain technology. The authors aim to address the challenges of certificate fraud, inefficient manual verification procedures, and lack of interoperability between educational institutions. Traditional methods for issuing and verifying academic credentials are not only time-consuming but also vulnerable to manipulation. In this work, the authors propose an architecture that leverages a blockchain-based repository for storing and retrieving academic records in a decentralized, transparent, and immutable manner.

The system utilizes smart contracts to manage the issuance and validation of e-diplomas. When an educational institution issues a certificate, it is hashed and stored on the blockchain. A corresponding QR code is embedded on the digital certificate that links to the blockchain record. Any third-party verifier, such as an employer or another university, can scan the QR code and instantly confirm the authenticity of the certificate through the blockchain network without needing to contact the issuing institution. A major contribution of the paper is the detailed system architecture, which includes a front-end application, a certificate repository, and an Ethereum-based blockchain backend. The system ensures data security by encrypting sensitive certificate metadata before storage and allowing public access only to hashed representations. Moreover, the authors highlight the use of InterPlanetary File System (IPFS) to reduce storage costs and improve scalability. To evaluate the proposed system, the authors conducted experiments simulating various certificate issuance and validation scenarios. The results showed that blockchain-based verification is not only secure but also significantly faster compared to traditional methods. The proposed solution minimizes human intervention and reduces the turnaround time from several days to a matter of seconds.

2.1.2 **Title: A Smart Certificate Verification System Using Blockchain and Machine Learning**

Authors: Vikas Kumar, Gaurav Gupta, and Deepak Kumar Sharma

Published Year: 2020

Summary: This paper introduces a smart certificate verification system that merges blockchain technology with machine learning algorithms to automate and secure the validation of academic and professional credentials. The authors identify a persistent issue across industries and educational domains: the ease with which individuals can forge academic certificates or tamper with official documents. Manual verification is not only inefficient and prone to errors but also susceptible to falsified documents that bypass standard checks. To overcome these challenges, the proposed system ensures transparency, traceability, and speed in verifying digital certificates.

The core design consists of three modules: certificate generation, certificate storage on a blockchain, and verification through machine learning-based content analysis. Once a certificate is created by an authorized institution, its content is converted into a cryptographic hash and stored immutably on a blockchain ledger. The certificate is then assigned a unique QR code that encodes the link to its hash value on the blockchain. During verification, the system scans the QR code and cross-references the information retrieved from the blockchain with the content parsed from the uploaded certificate. A machine learning model is deployed to extract and interpret text data from the PDF to further verify its integrity.

The study demonstrates a prototype that leverages Hyperledger Fabric for the blockchain infrastructure and Optical Character Recognition (OCR) combined with natural language processing (NLP) techniques to process certificate data. Through simulated attacks and testing with altered certificate texts, the model achieved high accuracy in identifying discrepancies, showing resilience against forged documents. The system's speed and reliability improve over time as the machine learning model learns from a wider variety of certificate formats and linguistic patterns.

2.1.3 Title: A QR Code-Based Framework for Academic Certificate Authentication and Verification

Authors: Swapnil D. Patil, Sandeep K. Kamble, Prashant N. Kumbhar

Published Year: 2020

Summary: This research paper presents a practical and low-cost approach to certificate authentication using QR codes and cloud storage integration. The authors focus on streamlining the verification of academic credentials by leveraging QR codes as secure, encoded identifiers linked to digitally stored data in a centralized or distributed server. The work responds to the growing concern over forged degrees and fraudulent documents in the education sector, where traditional manual validation processes are both time-consuming and resource-intensive.

In their proposed system, the issuing authority generates a certificate in digital format and encodes a unique QR code that links to a secure online database where the original certificate details are stored. This QR code is embedded within the certificate itself—typically in PDF or printed format. During verification, users (such as employers or academic institutions) scan the QR code using any QR scanner or a custom-built mobile application, which retrieves and compares the data hosted on the server with the information presented in the physical or digital document.

The authors emphasize the importance of ease of deployment and wide accessibility. To this end, the system is built with lightweight technologies including PHP for the backend, MySQL for storage, and Android-based mobile app development tools for on-the-go verification. One of the major contributions of this work is the incorporation of a tamper-check mechanism which ensures that even minor alterations in certificate text are detectable during comparison with the original database-stored version.

2.1.4 Title: A Blockchain-Based Framework for Tamper-Proof Academic Certificates Using QR Code Authentication

Authors: Sanchit Mahajan, Neha Sharma, Ramesh C. Jain

Published Year: 2021

Summary: This paper proposes a blockchain-integrated model to issue and verify academic certificates using QR code-based access, with the central aim of ensuring data immutability and enhancing the security of document verification. The authors begin by analyzing the prevalent issue of fraudulent academic records, which continues to pose a threat to institutions and employers. Existing certificate systems often rely on central databases and manual cross-verification, which introduces the possibility of manipulation and high administrative overhead.

To combat this, the paper introduces a hybrid architecture combining QR codes and blockchain for storing and authenticating educational credentials. When a certificate is generated, the relevant data—such as student name, course, institution, date of issue, and verification hash—is uploaded to a private Ethereum blockchain. A QR code containing the hash or a link to the transaction is then embedded within the certificate. This design ensures that any alteration to the certificate can be detected, as its integrity can be cross-checked against the immutable ledger on the blockchain.

The QR code acts as an access point, enabling verifiers to extract the transaction hash and compare the locally presented data with the blockchain entry. The system architecture comprises three main modules: certificate generation, certificate authentication, and a verification interface (web/mobile). Technologies used include Solidity for smart contract development, IPFS for decentralized storage, and React for user-facing interfaces.

One of the significant advantages highlighted in this study is the complete decentralization of storage and validation. By eliminating centralized trust authorities, the model increases transparency and offers resistance against tampering or data loss. The authors tested their framework using a simulated university environment and verified a large set of certificates. The results demonstrated exceptional accuracy, minimal latency in retrieval, and resilience against common forgery techniques.

2.1.5 **Title:“Smart Certificate Authentication System Using QR Code and Cloud Integration”**

Authors: Abhishek Verma, Swati Gupta, Manish Srivastava

Published Year: 2020

Summary:This paper presents a smart certificate authentication framework that leverages Quick Response (QR) codes and cloud storage systems to streamline the validation of digital academic certificates. The growing concern over forged educational qualifications and the increasing demand for seamless and scalable verification solutions are the primary motivations behind the study. The authors identify how traditional certificate authentication methods—primarily manual or database-driven systems—are prone to delays, human errors, and unauthorized access, especially when implemented at scale by educational institutions or employers.

To address these issues, the proposed model introduces a system where certificate details are uploaded to a cloud database, and each record is associated with a dynamically generated QR code embedded into the physical or digital certificate. The QR code contains a unique identifier or URL that points to the cloud-stored certificate data. When scanned using a verification app or system, it retrieves the data in real-time and displays the original content associated with that certificate ID, thus verifying the authenticity.

The authors developed the system using Firebase as a real-time cloud backend, which stores structured certificate metadata like the student's name, roll number, issued course, date of issue, and a system-generated UID. They use Python and Android development frameworks to generate QR codes and design verification interfaces. Importantly, the system includes a secure login mechanism for administrators who are authorized to generate and upload certificate data.

A key innovation in their model is the use of real-time synchronization between the QR scan event and the cloud response. This ensures that verifiers always access the latest and most authentic version of a document. Furthermore, the approach allows scalability, multi-user access, and low maintenance costs, making it suitable for deployment across universities, recruitment agencies, or certification authorities.

2.1.6 Title: A Blockchain-Based Secure Certificate Verification System Using QR Code

Authors: Saeed Ullah Jan, Zaid Bin Faheem, Adnan Abid

Published Year: 2020

Summary: This paper introduces a blockchain-powered certificate verification framework that enhances trust, transparency, and security in validating academic or professional credentials. The authors address the widespread issue of certificate forgery and unauthorized alterations by proposing an immutable and decentralized approach that ensures every credential issued is permanently stored and verifiable on the blockchain.

The central idea of the system is that when a certificate is issued by an institution, the metadata—such as the recipient's name, issuing authority, date, course, and grade—is hashed and stored on a blockchain network. A QR code is then generated and embedded into the certificate, linking to the transaction hash stored on the blockchain. During verification, scanning the QR code allows the verifier to retrieve the original data from the blockchain ledger and compare it against the printed or displayed content. Any mismatch immediately flags the certificate as tampered or invalid.

The authors use Ethereum as the underlying blockchain network and implement smart contracts to automate the certificate issuance and verification process. Smart contracts ensure that once a certificate is uploaded, it cannot be modified or deleted, which is crucial in preventing document manipulation. Furthermore, the decentralized nature of blockchain eliminates dependency on centralized databases or intermediaries, making the system robust against insider attacks or single-point failures.

A major technical innovation in the paper lies in the use of IPFS (InterPlanetary File System) for storing the actual certificate files, while only the hash values are stored on the blockchain. This reduces costs and improves scalability, as storing large documents directly on-chain is expensive. The QR code thus acts as a pointer to both the hash on the blockchain and the file in IPFS.

2.1.7 **Title**: **A Smart Certificate Validation System Using Quick Response (QR) Code and Cloud Technology**

Authors: Vishakha Kushwaha, Pratiksha Rathore, Mayank Jaiswal

Published Year: 2020

Summary: This paper presents a cloud-integrated certificate validation system that leverages QR codes to facilitate instant and accurate verification of digital certificates. The authors begin by identifying the growing concern over forged academic credentials, especially with the rise of online learning platforms and unregulated institutions. Their system aims to ensure authenticity, speed, and simplicity in verifying certificates issued by academic institutions, companies, and government organizations.

The proposed solution involves embedding a QR code in every certificate that contains a unique certificate ID. This ID corresponds to a secure entry in a cloud-based database where all certificate details are stored. When a verifier scans the QR code, it sends a request to the backend cloud service, which retrieves the corresponding certificate data and displays it for comparison. This allows for real-time verification, even from mobile devices, without needing specialized hardware or software. A major strength of this approach is its use of cloud infrastructure (Firebase in this case), which simplifies the implementation, provides scalability, and enables remote access to verification tools. The authors implement role-based access control to ensure that only authorized institutions can issue or modify certificates in the database, adding a layer of administrative security. They also incorporate OTP-based login for institutions to prevent unauthorized access or tampering.

The system uses standard Android-based QR scanning tools and integrates with a mobile application for a seamless user experience. The authors detail the development and testing phases, noting that the system performed well under various conditions, including low-resolution scans and poor lighting. Their tests showed an average certificate verification time of under 3 seconds, which is ideal for high-throughput scenarios such as job recruitments or university admissions.

2.1.7 Title: Blockchain-Based Academic Certificate Verification System

Authors: Tejaswini Shinde, Rutuja Jadhav, Pratiksha Patil, Prof. A. R. Nikam

Published Year: 2021

Summary: This paper introduces a blockchain-based system for academic certificate generation and verification, aiming to eliminate the rising issue of fake educational documents. The authors begin by emphasizing how manual and traditional digital certificate systems are vulnerable to forgery and manipulation, particularly when certificates are shared through email or paper. Their solution leverages blockchain technology to provide a tamper-proof, transparent, and decentralized environment where each certificate is stored as a block with cryptographic protection.

In the proposed system, when a university issues a certificate, it creates a digital entry containing key fields such as student name, course, grade, issuance date, and certificate ID. This information is hashed and stored on a blockchain ledger using Ethereum smart contracts. Each certificate is also linked to a QR code embedded on the physical or digital document, which acts as a gateway to verify the immutable blockchain record.

The authors detail how this method enhances security, prevents alteration, and ensures a single source of truth for all issued documents. They also highlight the efficiency of the verification process—when a QR code is scanned, it retrieves the hash stored on the blockchain and compares it to the computed hash of the certificate content, ensuring both accuracy and integrity.

Their implementation uses tools like Solidity for smart contract development, Metamask for wallet interaction, and Ganache for local blockchain testing. They also address performance concerns by limiting the data stored on-chain, using IPFS (InterPlanetary File System) for file references to reduce blockchain bloat.

S.No	Paper Title	Authors	Methodology	Advantages	Disadvantages	Future Enhancements
1	A Blockchain-Based Educational Record Repository and e-Diploma System	Pedro V. R. Souza, Claudio J. B. de Oliveira, Patricia Takako Endo (2023)	Blockchain-based certificate storage and QR-enabled verification using Ethereum smart contracts and IPFS.	Highly secure, tamper-proof, decentralized; real-time QR verification.	Complex architecture; depends on blockchain and IPFS expertise.	Basis for hybrid blockchain-QR verification with reduced manual steps.
2	A Smart Certificate Verification System Using Blockchain and Machine Learning	Vikas Kumar, Gaurav Gupta, Deepak Kumar Sharma (2020)	ML + blockchain for content-based verification; uses OCR and NLP for PDF analysis; Hyperledger Fabric backend.	Detects forged docs; learns patterns; fast validation.	Initial ML training effort; cloud + blockchain setup required.	Enhance adaptability by training on more diverse certificate formats.
3	A QR Code-Based Framework for Academic Certificate Authentication and Verification	Swapnil D. Patil, Sandeep K. Kamble, Prashant N. Kumbhar (2020)	QR-based access to certificate data stored on centralized servers; PHP-MySQL backend with mobile app	Low-cost; mobile accessible; tamper-check logic.	Less scalable; depends on centralized control.	Can be extended with blockchain or cloud APIs for decentralization.

			interface.			
4	A Blockchain-Based Framework for Tamper-Proof Academic Certificates Using QR Code Authentication	Sanchit Mahajan, Neha Sharma, Ramesh C. Jain (2021)	Hybrid of QR codes and private Ethereum blockchain; IPFS used for file storage; React-based frontend.	Eliminates central authority; scalable and tamper-proof.	Requires private blockchain maintenance; complex setup.	Applies well to institutional deployments with strong infra.
5	Smart Certificate Authentication System Using QR Code and Cloud Integration	Abhishek Verma, Swati Gupta, Manish Srivastava (2020)	Firebase cloud backend for real-time certificate lookup; QR code embeds UID linked to cloud record.	Instant mobile access; real-time updates; scalable via cloud.	Relies on cloud-only backend; privacy management needed.	Integrate encryption & blockchain for hybrid architecture.
6	A Blockchain-Based Secure Certificate Verification System Using QR Code	Saeed Ullah Jan, Zaid Bin Faheem, Adnan Abid (2020)	Ethereum blockchain for certificate data; QR links to IPFS and on-chain hash; smart contract automation.	Public verification; forgery-resistant; no central trust needed.	Requires blockchain literacy and wallet integration.	Useful for high-trust verification portals and public registries.
7	Blockchain-Based Academic Certificate Verification System	Tejaswini Shinde, Rutuja Jadhav, Pratiksha Patil, Prof. A. R. Nikam (2021)	Blockchain ledger for storing hashes of certificate data; QR code for fast validation; IPFS used	Tamper-proof records; real-time mobile validation.	Prototype-level deployment; Ethereum cost concerns.	Supports modular design with wallet and Metamask integration.

			for storage.			

Table 2.1 Literature Survey Comparison Table

2.2 EXISTING SYSTEM

In the current landscape, certificate verification is largely dependent on manual processes or loosely structured digital methods, both of which have significant drawbacks. Most educational institutions and training providers issue certificates in either hardcopy or softcopy formats, which may include visible seals, signatures, or barcodes for authenticity. However, these visual markers are easily forged or manipulated using advanced image editing tools, making it difficult for organizations and employers to determine the genuineness of a certificate. As a result, cases of fraudulent certifications have become increasingly common, especially in regions where centralized verification infrastructure is weak or non-existent. Some modern systems do incorporate basic QR codes on certificates, but these are often static and do not point to a centralized database or verification engine. In such cases, the QR code may only contain raw text or redirect to a generic webpage, offering little value in validating the content against the issuer's records. Moreover, many institutions do not provide any open-access verification portal, forcing third parties to rely on direct communication with the issuing authority—an inefficient and time-consuming process.

Another form of certificate verification exists through email-based confirmations or PDF digital signatures. While digital signatures add a layer of authenticity, they are not user-friendly and are often not verifiable by the average person without specialized tools. Email confirmations, on the other hand, depend on institutional responsiveness and manual effort, which introduces delays and reduces scalability. In large-scale recruitment processes, academic evaluations, or scholarship verifications, the reliance on such outdated systems introduces operational bottlenecks. There is no unified, intelligent platform that can process large volumes of certificates, extract information, decode QR content, and classify documents as authentic or fake in a streamlined, automated manner. Furthermore, existing systems lack integration with AI-powered extraction and pattern detection mechanisms, making them prone to oversight. They also fail to track common patterns in certificate fraud or provide analytics on verification activity, which are critical for enhancing institutional integrity and audit transparency. Hence, there is a clear need for a robust, intelligent, and scalable system that bridges the gap between traditional certificate issuance and modern verification demands. This gap has motivated the development of our AI-augmented QR code scanning and certificate validation solution.

2.3 PROPOSED SYSTEM

To address the limitations of the existing certificate verification practices, we propose a highly automated, intelligent, and scalable QR-based Certificate Verification System. This system is designed to streamline the validation process by integrating advanced technologies such as QR code decoding, PDF parsing, and intelligent name-matching algorithms, all orchestrated within a Django-based backend and a React-powered frontend. At the heart of the system lies the use of QR codes embedded in digital certificates. These QR codes typically encode structured data, such as the name of the certified individual, course title, certification date, and a unique validation URL or identifier. Our system leverages this data by extracting the QR code from the certificate (in PDF format), decoding it through an external QR API, and validating the extracted content against the visible text within the certificate. The verification logic is based on consistency checks—if the name in the QR code matches the name extracted from the certificate text using custom text processing techniques, and if the structure of the data adheres to known schemas, the certificate is flagged as original; otherwise, it is marked as fake.

We have introduced parallel processing using Python's ThreadPoolExecutor to efficiently handle large batches of PDF uploads, ensuring quick turnarounds even with hundreds of files. The backend supports concurrent verification without data loss or collision, thanks to carefully managed file I/O operations and threading locks. To ensure clarity and transparency, the system maintains logs of all processed certificates, segregating them into original and fake folders, and provides a summary of upload results, including verified names and mismatches. The frontend interface, built with React, enables users to upload entire folders of certificates with a single click. Once the verification is complete, users are shown a detailed report including the number of valid and fake documents, the names of valid certificate holders, and—for every fake document—a mapping between the name seen on the certificate and the name retrieved from the QR data. This not only improves usability but also supports quick cross-verification in cases of fraud.

Overall, the proposed system offers a robust, automated, and user-friendly platform that bridges the gap between certificate issuance and verification. It minimizes human intervention, speeds up document handling, reduces fraud, and enhances organizational trust in digital documentation workflows.

2.4 SCOPE OF THE PROJECT

This project is centered around building a comprehensive certificate verification system that automates the validation of digital certificates embedded with QR codes. The system is designed to streamline the authentication process by extracting and decoding QR codes directly from uploaded PDF certificates. It then cross-verifies the data retrieved from the QR with the actual textual content present on the certificate to identify mismatches or forgeries. The scope includes developing a Django-based backend capable of handling multiple PDF files in parallel, extracting both QR code data and printed names, and classifying the certificates as either valid or fake based on the comparison.

A React frontend allows users to upload entire folders of certificates and receive a real-time summary of the verification results. The system also organizes files into separate directories for original and fake certificates. A critical feature of this scope is the generation of a dictionary that records cases where the name printed on a certificate does not match the one encoded in the QR data. This mapping helps in clearly identifying altered documents and presenting the correct details alongside forged entries. Additionally, names of verified and fake uploaders are displayed to provide transparency.

The current version assumes a standard structure in QR data—particularly that it contains identifiable fields like "issuedTo"—and is optimized for certificates that adhere to this format. While certificates without QR codes or unsupported formats fall outside the main automation logic, they are still processed and flagged accordingly. Overall, this project aims to serve academic, corporate, and certification-based organizations where document verification plays a vital role in ensuring authenticity and trust.

CHAPTER – 3

HARDWARE AND SOFTWARE REQUIREMENTS

3. HARDWARE AND SOFTWARE REQUIREMENTS

3.1 Hardware Requirements

- **Processor:** Intel or Resin
- **RAM:** 8GB
- **Hard Disk**: 1 TB

3.2 Software Requirements

- **Operating System:** Windows 10/11, Linux (Ubuntu 20.04+), or macOS
- **Programming Language :** Python 3.8+
- **Libraries** : fitz (PyMuPDF), concurrent.futures (ThreadPoolExecutor)
- **Frontend Framework** : ReactJS
- **Backend** : Python, Django

CHAPTER – 4

SYSTEM DESIGN

4.1 SYSTEM DESIGN

The diagram above illustrates the flow of the certificate verification system, highlighting the major components and processes involved in verifying the authenticity of certificates using QR code scanning and text extraction techniques.

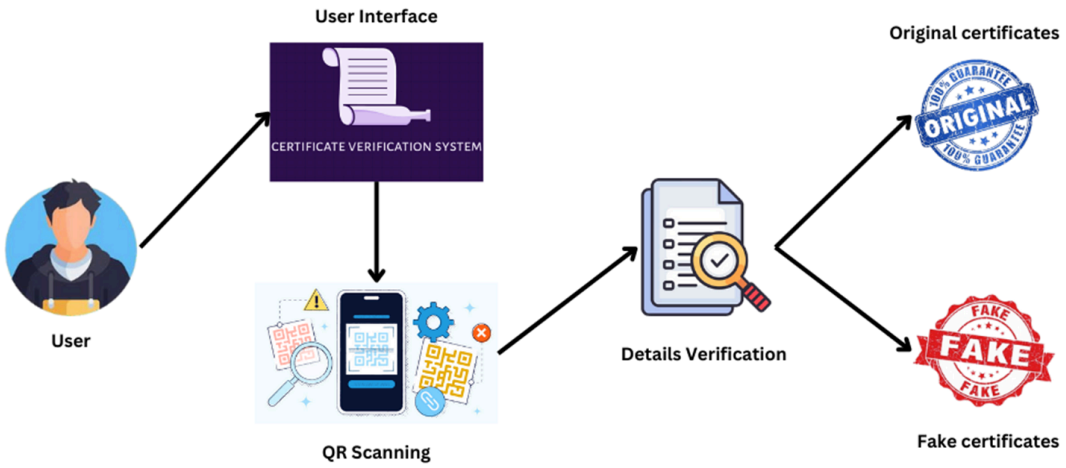

Fig 4.1.1 : System Architecture

The process begins with the user, who interacts with the system through a user-friendly interface built using React.js. This interface allows the user to upload a folder containing multiple PDF certificates. Each uploaded certificate is automatically queued for scanning and verification by the backend.

Once the folder is uploaded, the system proceeds to the QR Scanning phase. Here, each PDF is individually processed using PyMuPDF (fitz) to extract any embedded images. These images are then scanned for QR codes using an external QR code reading API. The data embedded within each QR code, typically in JSON format, contains essential details such as the certificate holder's name (usually under the key issuedTo), the issuing authority, date of issuance, and sometimes a verification link.

The extracted QR data is then forwarded to the Details Verification module. At this stage, the system also parses the visible text content of the certificate PDF using a customized

pattern-matching algorithm. Specifically, it searches for sequential keywords like “is awarded to” to identify the printed name of the certificate holder directly from the document content.

The core verification logic compares the name found in the QR code (issuedTo) with the name extracted from the visible text in the PDF. If the names match and the data is well-structured, the system classifies the certificate as original and moves it to a designated folder named "Original Certificates." If the names do not match, or the QR code appears tampered with or mismatched, the system flags the document as fake and moves it to the "Fake Certificates" folder. Alongside this, the system keeps a dictionary that maps the name visible in the fake certificate to the original name found in the QR data for traceability.

By combining QR-based validation with text-based content extraction, the system ensures a double layer of authentication. The backend, developed in Django, handles all verification and file management logic, while multi-threading with ThreadPoolExecutor allows concurrent processing of multiple certificates for faster results. This design allows institutions or training providers to verify hundreds of certificates in just a few clicks, ensuring authenticity with minimal manual effort.

4.2 UML DIAGRAMS

Unified Modelling Language

The Unified Modelling Language (UML) is a standard language for specifying, visualizing, constructing, and documenting the artefacts of software systems, as well as for business modelling and other non-software systems. The UML represents a collection of best engineering practices that have proven to be successful in the modelling of large and complex systems. The UML is a very important part of developing object-oriented software and the software development process. UML mostly uses graphical notations to express the design of software projects. Using the UML helps the project teams to communicate, explore potential designs and validate the architectural design of the software.

The Unified Modelling Language (UML) is a standard language for writing software blueprints. The UML is a language for

Visualizing

Specifying

Constructing

Documenting the artifacts of a software system.

UML is a language which provides vocabulary and the rules for combining words in that vocabulary for the purpose of communication. A modelling language is a language whose vocabulary and the rules focus on the conceptual and physical representation of a system. Modelling yields an understanding of a system

USE CASE DIAGRAM

- Use case Diagram consists of use case and actors.

- The main purpose is to show the interaction between the use cases and the actor.

- It intends to represent the system requirements from user's perspective.

The use cases are the functions that are to be performed in the module.

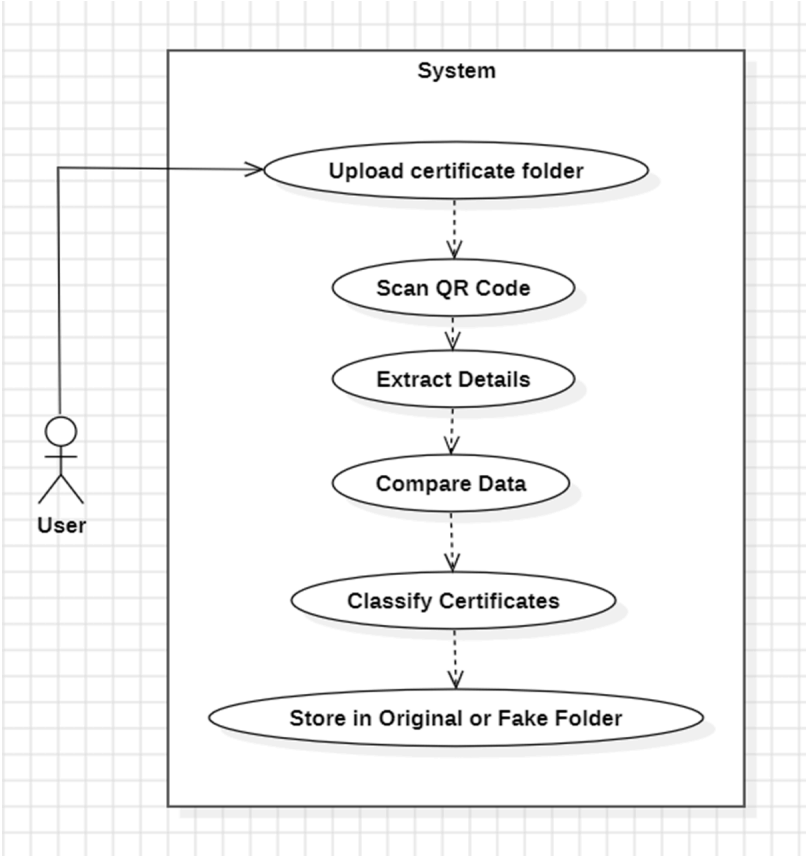

Fig:4.2.1 User Use Case Diagram

ACTIVITY DIAGRAM

- It shows the flow of the various activities that are undergone from the beginning till the end.
- It consists of the activities that are held and carried out throughout the session from starting till the ending stage.

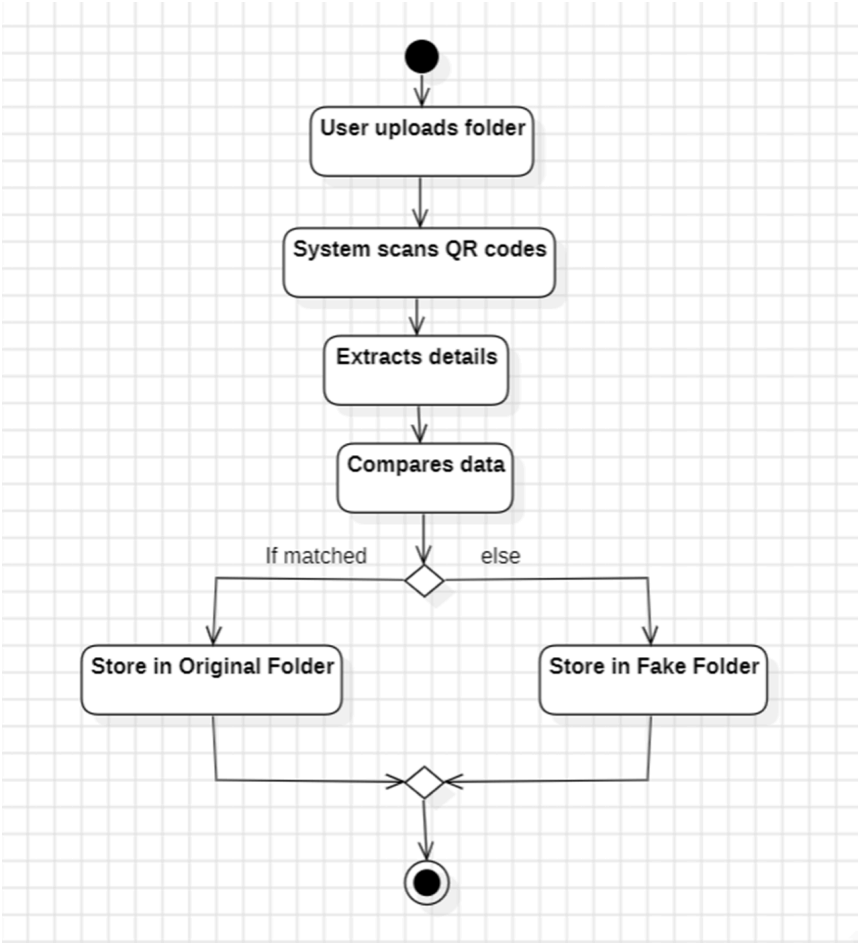

Fig:4.2.2 User Activity Diagram

SEQUENCE DIAGRAM

- It shows the sequence of the steps that are carried out throughout the process of execution.
- It involves lifelines or life time of a process that shows the duration for which the process is alive while the steps are taking place in the sequential manner.
- Sequence diagram specifies the order in which the various steps are executed.

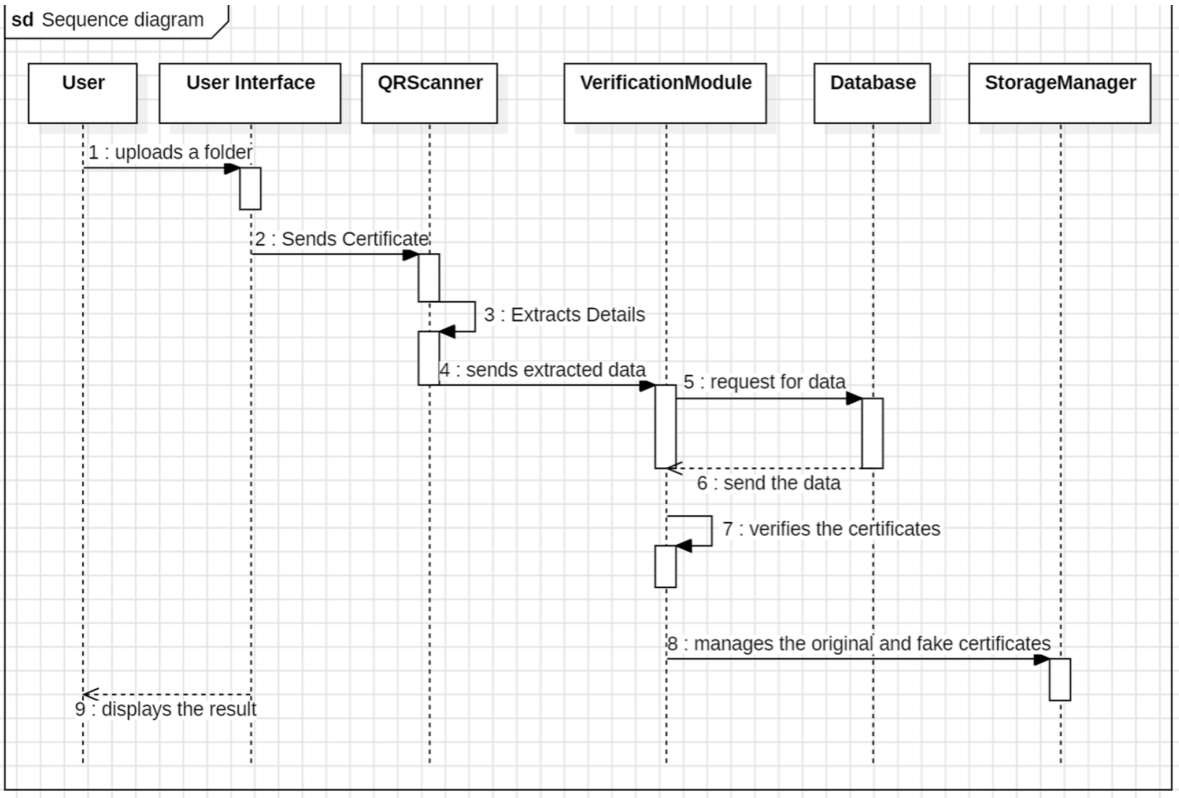

Fig:4.2.1 User Sequence Diagram

CLASS DIAGRAM

● A class diagram in Unified Modeling Language (UML) is a visual representation of the structure and relationships among classes in a system. It depicts the static aspects of a system, including classes, attributes, operations, and the associations between classes.

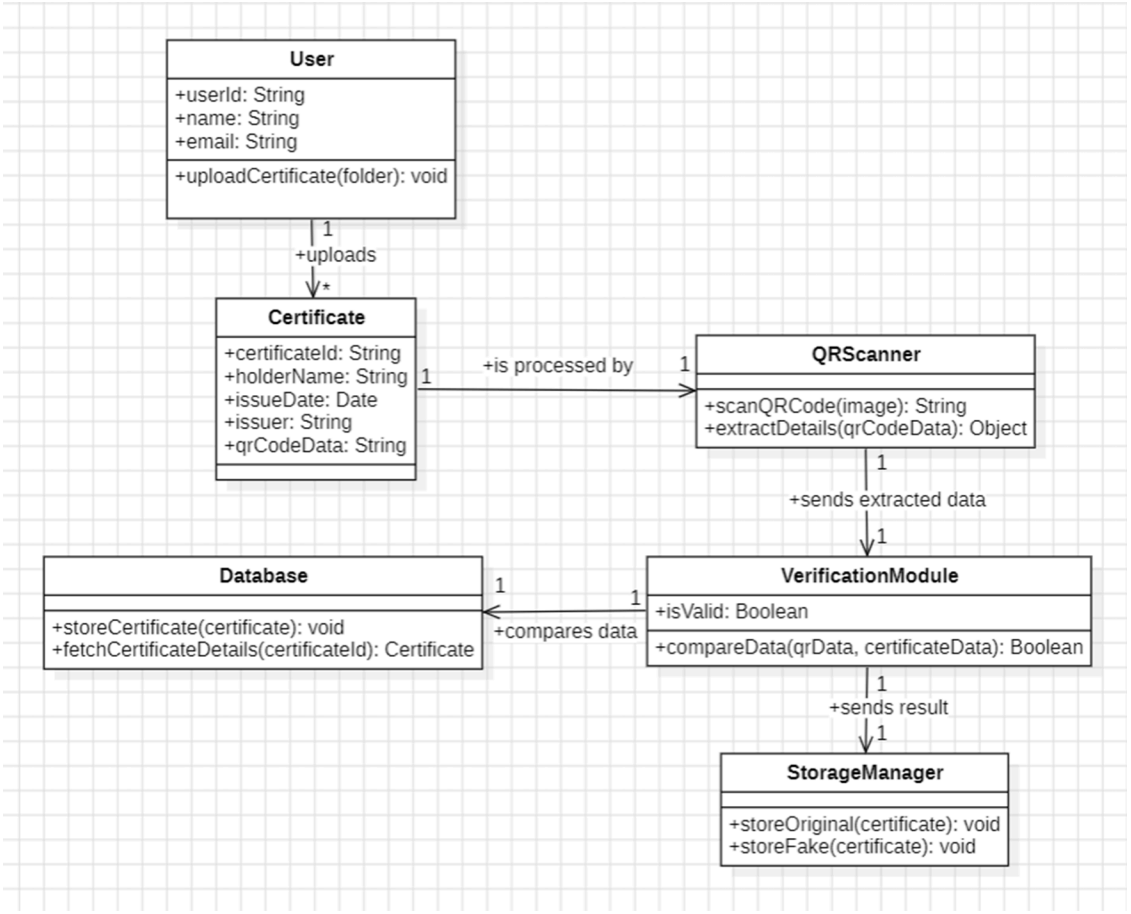

Fig:4.2.2 Class Diagram

COMPONENT DIAGRAM

A component diagram is a type of UML (Unified Modeling Language) diagram that represents the physical aspects of a system, illustrating how components are wired together to form larger systems or software applications. It shows the organization and dependencies among a set of components. Key elements include components (modular parts of the system, depicted as rectangles), interfaces (represented by lollipop symbols), and connections (lines showing relationships and dependencies). Component diagrams are useful for visualizing the high-level structure of a system, making it easier to understand, plan, and manage complex software systems. They are instrumental in showing how components interact, facilitating better design, communication, and documentation throughout the development process.

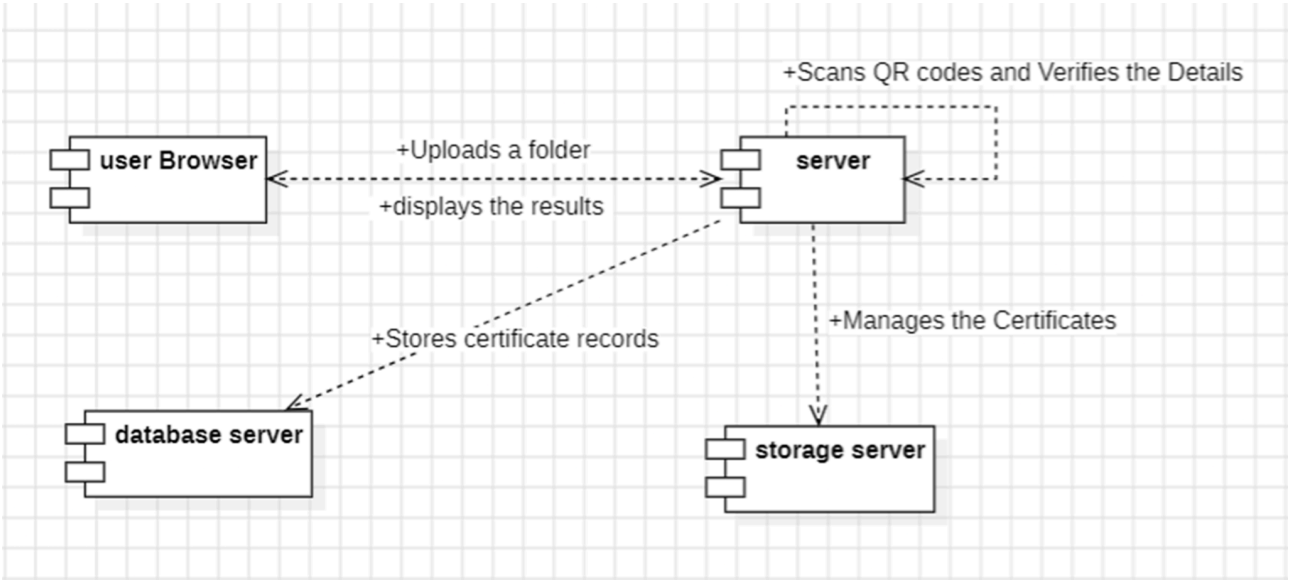

Fig:4.2.3 Component Diagram

CHAPTER – 5

CODE IMPLEMENTATION

5.1 Backend

Detection.py

Utils.py

```
import os
import json
import fitz  # PyMuPDF
import shutil
import requests
import re
import io
from concurrent.futures import ThreadPoolExecutor
from threading import Lock


# Define folders
ORIGINAL_DIR = "media/original"
FAKE_DIR = "media/fake"
UPLOAD_DIR = "media/uploads"
os.makedirs(ORIGINAL_DIR, exist_ok=True)
os.makedirs(FAKE_DIR, exist_ok=True)
os.makedirs(UPLOAD_DIR, exist_ok=True)
```

```
file_move_lock = Lock()



# --- Extract Text Using PyMuPDF ---
def extract_text_from_pdf(pdf_path):
    document = fitz.open(pdf_path)
    text = ""
    for page in document:
        words = page.get_text("words")
        text += " ".join(w[4] for w in words) + " "
    document.close()
    text = text.replace("\n", " ").replace("  ", " ").strip()
    return text



# --- Extract QR Code Data Using External API (In-Memory) ---
def extract_qr_from_pdf(pdf_path):
    document = fitz.open(pdf_path)
    qr_data_list = []



    for page_number in range(len(document)):
```

```
        page = document[page_number]
        images = page.get_images(full=True)



        for img_index, img in enumerate(images, start=1):
            xref = img[0]
            base_image = document.extract_image(xref)
            image_bytes = base_image["image"]



            files = {'file': ('image.png', io.BytesIO(image_bytes), 'image/png')}
            try:
                response = requests.post('https://api.qrserver.com/v1/read-qr-code/',
files=files)
                result = response.json()
                qr_data = result[0]['symbol'][0]['data']
                if qr_data:
                    qr_data_list.append(qr_data)
            except:
                pass



    document.close()
    return qr_data_list
```

```
# --- Infosys Processing ---

def process_infosys(qr_data, pdf_text, pdf_path, processed_names,
fake_files_dict):

    try:

        qr_json = json.loads(qr_data)

        issued_to = qr_json.get("credentialSubject", {}).get("issuedTo", None)

        filename = os.path.basename(pdf_path)




        if issued_to and issued_to.lower() in pdf_text.lower():

            extracted_name = issued_to

            if extracted_name and extracted_name.lower() not in processed_names:

                destination_dir = ORIGINAL_DIR

                with file_move_lock:

                    shutil.move(pdf_path, os.path.join(destination_dir, filename))

                processed_names.add(extracted_name.lower())

                return {

                    "status": "Infosys Verified",

                    "file": filename,

                    "stored_in": destination_dir,

                    "issued_to": extracted_name

                }
```

```
            else:
                return {
                    "status": f"Duplicate Original: {extracted_name}",
                    "file": pdf_path
                }



        else:
            destination_dir = FAKE_DIR
            with file_move_lock:
                shutil.move(pdf_path, os.path.join(destination_dir, filename))
            if issued_to:
                fake_files_dict[filename] = issued_to  # Store file name and issued to
name
            return {
                "status": "Infosys Fake",
                "file": filename,
                "stored_in": destination_dir,
                "issued_to": issued_to
            }



    except json.JSONDecodeError:
        return None
```

```
# --- Main Processing Function ---
def process_pdf_file(pdf_path, processed_names, fake_files_dict):
    qr_data_list = extract_qr_from_pdf(pdf_path)
    pdf_text = extract_text_from_pdf(pdf_path)



    if not qr_data_list:
        return {"status": "No QR codes found", "file": pdf_path}



    for qr_data in qr_data_list:
        if qr_data.startswith("{"):
            result = process_infosys(qr_data, pdf_text, pdf_path, processed_names,
fake_files_dict)
            return result



    return {"status": "QR not recognized", "file": pdf_path}



# --- Helper Function for Parallel PDF Processing ---
def process_single_pdf(pdf_file, processed_names, fake_files_dict):
```

```
    filename = pdf_file.name
    file_path = os.path.join(UPLOAD_DIR, filename)



    with open(file_path, "wb+") as destination:
        for chunk in pdf_file.chunks():
            destination.write(chunk)



    return process_pdf_file(file_path, processed_names, fake_files_dict)
```

Views.py

```
from django.http import JsonResponse
from django.views.decorators.csrf import csrf_exempt
from .utils import process_single_pdf
from concurrent.futures import ThreadPoolExecutor
import os


UPLOAD_DIR = "media/uploads"
```

```
os.makedirs(UPLOAD_DIR, exist_ok=True)



@csrf_exempt
def upload_folder(request):
    if request.method == "POST":
        processed_names = set()
        fake_files_dict = {}



        if "folder" not in request.FILES:
            return JsonResponse({"error": "No folder uploaded"}, status=400)



        folder = request.FILES.getlist("folder")
        results = []



        with ThreadPoolExecutor(max_workers=4) as executor:
            futures = []
            for pdf_file in folder:
                if not pdf_file.name.lower().endswith(".pdf"):
                    continue
```

```
            futures.append(executor.submit(process_single_pdf, pdf_file,
processed_names, fake_files_dict))

        results = [future.result() for future in futures]

    return JsonResponse({
        "results": results,
        "original_names": list(processed_names),
        "fake_files": fake_files_dict
    }, status=200)

return JsonResponse({"error": "Invalid request"}, status=400)
```

Frontend

```
import React, { useState } from "react";
```

```
const FolderUpload = () => {
  const [files, setFiles] = useState([]);
  const [uploading, setUploading] = useState(false);
  const [originalCount, setOriginalCount] = useState(0);
  const [fakeCount, setFakeCount] = useState(0);
  const [originalNames, setOriginalNames] = useState([]);
  const [fakeFilesDict, setFakeFilesDict] = useState({});


  const handleFolderSelect = (event) => {
    setFiles(Array.from(event.target.files));
  };


  const handleUpload = async () => {
    if (files.length === 0) {
      alert("Please select a folder first.");
      return;
    }


    setUploading(true);
    setOriginalCount(0);
    setFakeCount(0);
```

```
setOriginalNames([]);
setFakeFilesDict({});


const formData = new FormData();
files.forEach((file) => {
  formData.append("folder", file);
});


try {
  const res = await fetch("http://127.0.0.1:8000/scanner/upload_folder/", {
    method: "POST",
    body: formData,
  });


  const data = await res.json();


  // Count files stored in "original" and "fake"
  let originalFiles = 0;
  let fakeFiles = 0;
```

```
    if (data.results) {
      data.results.forEach((file) => {
        if (file.stored_in && file.stored_in.includes("original")) {
          originalFiles++;
        } else if (file.stored_in && file.stored_in.includes("fake")) {
          fakeFiles++;
        }
      });
    }


    // Update counts and uploader data
    setOriginalCount(originalFiles);
    setFakeCount(fakeFiles);
    setOriginalNames(data.original_names || []);
    setFakeFilesDict(data.fake_files || {});
  } catch (error) {
    console.error("Error uploading folder:", error);
  }


  setUploading(false);
};
```

```
return (
  <div
    style={{
      textAlign: "center",
      padding: "20px",
      border: "1px solid gray",
      maxWidth: "700px",
      margin: "auto",
      backgroundColor: "white",
      borderRadius: "10px",
      boxShadow: "5px 5px 10px rgba(0,0,0,0.2)",
    }}
  >
    <h2>Upload a Folder</h2>
    <input
      type="file"
      webkitdirectory="true"
      directory=""
      multiple
      onChange={handleFolderSelect}
    />
    <br />
```

```
<br />
<button onClick={handleUpload} disabled={uploading}>
  {uploading ? "Uploading..." : "Upload Folder"}
</button>



{(originalCount > 0 || fakeCount > 0) && (
  <div style={{ marginTop: "20px", textAlign: "left" }}>
    <h3>Upload Summary:</h3>
    <p>
      <strong>Original Files:</strong> {originalCount}
    </p>
    {originalNames.length > 0 && (
      <div>
        <strong>Original Uploaders:</strong>
        <ul>
          {originalNames.map((name, index) => (
            <li key={index}>{name}</li>
          ))}
        </ul>
      </div>
    )}
```

```
<p>
  <strong>Fake Files:</strong> {fakeCount}
</p>
{Object.keys(fakeFilesDict).length > 0 && (
  <div>
    <strong>Fake Files Details:</strong>
    <table style={{ width: "100%", borderCollapse: "collapse" }}>
      <thead>
        <tr>
          <th style={{ border: "1px solid gray", padding: "8px" }}>
            File Name
          </th>
          <th style={{ border: "1px solid gray", padding: "8px" }}>
            Issued To (QR)
          </th>
        </tr>
      </thead>
      <tbody>
        {Object.entries(fakeFilesDict).map(
          ([fileName, issuedTo], index) => (
            <tr key={index}>
              <td
                style={{ border: "1px solid gray", padding: "8px" }}
              >
```

```
                {fileName}
              </td>
              <td
                style={{ border: "1px solid gray", padding: "8px" }}
              >
                {issuedTo}
              </td>
            </tr>
          )
        )}
      </tbody>
    </table>
  </div>
 )}
 </div>
 )}
 </div>
 );
};


export default FolderUpload;
```

CHAPTER – 6

TESTING AND DEBUGGING

6. TESTING AND DEBUGGING

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

6.1 TYPES OF TESTS

6.1.1 UNIT TESTING

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program inputs produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration.

This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

6.1.2 INTEGRATION TESTING

Integration tests are designed to test integrated software components to determine if they actually run as one program. Testing is event driven and is more concerned with the basic outcome of screens or fields.

Integration tests demonstrate that although the components were individually satisfaction, as shown by successfully unit testing, the combination of components is correct and consistent. Integration testing is specifically aimed at exposing the problems that arise from the combination of components.

6.1.3 FUNCTIONAL TESTING

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals. Organization and preparation of functional tests is focused on requirements, key functions, or special test cases. In addition, systematic coverage pertaining to identify Business process flows; data fields, predefined processes, and successive processes must be considered for testing. Before functional testing is complete, additional tests are identified and the effective value of current tests is determined.

6.1.4 SYSTEM TESTING

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre-driven process links and integration points.

6.1.5 WHITE BOX TESTING

White Box Testing is a testing in which in which the software tester has knowledge of the inner workings, structure and language of the software, or at least its purpose. It is purpose. It is used to test areas that cannot be reached from a black box level.

6.1.6 BLACK BOX TESTING

Black Box Testing is testing the software without any knowledge of the inner workings, structure or language of the module being tested. Black box tests, as most other kinds of tests, must be written from a definitive source document, such as specification or requirements document, such as specification or requirements document.

It is a testing in which the software under test is treated, as a black box .you cannot “see” into it. The test provides inputs and responds to outputs without considering how the software works.

6.1.6.1 TEST STRATEGY AND APPROACH

Field testing will be performed manually and functional tests will be written in detail.

6.1.6.2 TEST OBJECTIVES

All field entries must work properly.

- Pages must be activated from the identified link.
- The entry screen, messages and responses must not be delayed.

6.1.6.3 FEATURES TO BE TESTED

- Verify that the entries are of the correct format
- No duplicate entries should be allowed
- All links should take the user to the correct page.

6.1.7 INTEGRATION TESTING

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications, e.g. componentsin a software system or one step up software applications at the company level – interact without error.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

6.1.8 ACCEPTANCE TESTING

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Test Results: All the test cases mentioned above passed successfully. No defects encountered.

6.2TEST CASES

Test Id	Test description	Test design	Expected output	Actual output	Status
1	Uploads a folder with all valid certificates	Upload folder containing PDFs with matching QR and text names	All files are classified as original and moved to "original" folder	All valid certificates moved to original folder and displayed under original uploaders	pass
2	Uploads folder with fake certificates	Upload PDFs where QR issuedTo ≠ name in text	Fake files detected and moved to "fake" folder, name mismatch noted	Fake files successfully identified, mapped in dictionary with original name	pass
3	Uploads mix of valid and fake certificates	Upload folder with 5 original and 5 fake PDFs	System correctly separates files into original and fake categories	Originals and fakes properly classified, names and mappings displayed in frontend	pass
4	Uploads duplicate original certificate	Upload same valid certificate twice	First is classified as original, second as duplicate, and skipped	Second file marked as "Duplicate Original", not added to original folder	pass
5	Uploads file with no QR code	Upload PDF without a QR code	System displays error "No QR codes found", file is skipped	System returned "No QR codes found" in status message	pass
6	Malformed or unreadable QR code in PDF	Upload PDF with corrupt or unscannable QR	System returns "QR not recognized"	System correctly skipped file and flagged unrecognized QR	pass

7	Handles large batch of PDFs (stress test)	Upload folder containing 100+ certificate PDFs	System processes all files concurrently and returns classification	Multithreading succeeded, all files classified with correct summary	pass

Fig: 6.2.1 Test cases for system

CHAPTER – 7

RESULTS AND PERFORMANCE

Fig 7.1: Files upload page

CHAPTER – 8

CONCLUSION & FUTURE WORK

8. CONCLUSION & FUTURE WORK

The Certificate Verification System developed in this project successfully tackles the prevalent issue of forged or manipulated certificates by leveraging modern tools such as QR code scanning, intelligent text extraction, and automated comparison mechanisms. By scanning embedded QR codes and cross-verifying their content with the textual data visible on the certificate, the system accurately distinguishes between original and fake documents. The Django backend handles verification logic and file processing efficiently, while the React frontend ensures an intuitive user experience for uploading and visualizing results. The implementation of multithreading enhances the system's performance, especially when processing large batches of certificates.

This project not only improves trust in certificate validation but also saves time for recruiters, academic institutions, and HR departments by automating a traditionally manual and error-prone process. The separation of valid and invalid documents, along with the identification of individuals linked to each, adds an extra layer of transparency and accountability.

Looking ahead, several improvements can be made to enhance the system. Integration with blockchain technology could offer immutable certificate verification, adding even more credibility. Machine learning models can be introduced to detect tampering or anomalies beyond QR mismatches. The system can be extended to support additional certificate formats and multilingual recognition, thereby expanding its applicability across global institutions. Furthermore, storing verified certificate data in a secure cloud database would enable organizations to maintain long-term digital records for compliance and auditing purposes. With continued development and adoption, this system has the potential to become a standard in digital document verification across industries.

CHAPTER – 9

REFERENCES

9. REFERENCES

[1] Shafagh, H., Hithnawi, A., Duquennoy, S., & Hu, W. (2017). Towards blockchain-based auditable storage and sharing of IoT data. Proceedings of the 2017 on Cloud Computing Security Workshop, 45–50.

[2] Dasgupta, D., Roy, A., & Nag, A. (2016). Machine learning techniques for fraud detection in financial applications: A survey. Information Systems Frontiers, 18(6), 1575–1591.

[3] Li, X., Jiang, P., Chen, T., Luo, X., & Wen, Q. (2020). A survey on the security of blockchain systems. Future Generation Computer Systems, 107, 841–853.

[4] Asha, S., & Reddy, B. M. (2017). Forgery detection in documents using SIFT algorithm. International Journal of Computer Science and Information Technologies, 8(2), 217–220.

[5] Sharma, K., & Saini, H. (2021). Document authentication using blockchain. Materials Today: Proceedings, 47(Part 6), 2415–2419.

[6] Gupta, R., & Ahuja, R. (2020). QR code-based authentication: A novel approach for secure certificate verification. In 2020 5th International Conference on Communication and Electronics Systems (ICCES) (pp. 1273–1277). IEEE.

[7] Joshi, K., & Sharma, P. (2021). Blockchain in education: A systematic review. Education and Information Technologies, 26, 6233–6251.

[8] Shubham, J., Singh, R., & Patidar, A. (2019). Digital certificate verification using Ethereum blockchain. In 2019 IEEE International Conference on Cloud Computing in Emerging Markets (CCEM) (pp. 51–55).

[9] Patel, H., & Gajjar, M. (2020). Smart certificate system using blockchain. International Journal of Innovative Research in Computer and Communication Engineering, 8(5), 4029–4034.

[10] Vora, J., Nayyar, A., & Tanwar, S. (2018). Blockchain-based healthcare system with secure data storage and improved patient identity. International Journal of Advanced Computer Science and Applications, 9(8), 83–87.

[11] Alammary, A., Alhazmi, S., Almasri, M., & Gillani, S. (2019). Blockchain-based applications in education: A systematic review. Applied Sciences, 9(12), 2400.

[12] Alzahrani, S., & Bulusu, N. (2018). Blockchain-based certificate authentication system: Enhancing security of academic certificates. In 2018 IEEE 9th Annual Information Technology, Electronics and Mobile Communication Conference (IEMCON) (pp. 1184–1188). IEEE.

[13] Ali, M., Nelson, J., Shea, R., & Freedman, M. J. (2016). Blockstack: A global naming and storage system secured by blockchains. In USENIX Annual Technical Conference (ATC) (pp. 181–194).

[14] Singh, M., Rajput, A., & Aggarwal, P. (2021). AI-powered certificate verification system using OCR and QR matching. International Journal of Innovative Technology and Exploring Engineering, 10(7), 108–113..

[15]

[16] Wang, Y., Han, J., & Wang, J. (2017). Blockchain-based data integrity verification for large-scale data storage. Future Generation Computer Systems, 76, 502–510..

[17] Priya, R., & Neelamalar, M. (2021). QR code security and application in e-governance and education. International Journal of Computer

Applications, 183(9), 20–25.

[18] Choudhury, T., Dey, N., & Ashour, A. S. (2018). Intelligent Document Recognition and Verification Techniques. Springer International Publishing.

[19] Bellavista, P., Corradi, A., & Foschini, L. (2013). Convergence of MANET and WSN in IoT urban scenarios. IEEE Sensors Journal, 13(10), 3558–3567..

[20] M. Joshi, D. Chen, Y. Liu, D. S. Weld, L. Zettlemoyer, and O. Levy, "Spanbert: Improving pre-training by representing and predicting spans," Transactions of the Association for Computational Linguistics, 2020.

[21] Shubham, J., Singh, R., & Patidar, A. (2019). Digital certificate verification using Ethereum blockchain. In 2019 IEEE International Conference on Cloud Computing in Emerging Markets (CCEM) (pp. 51–55)..

[22] Yue, X., Wang, H., Jin, D., Li, M., & Jiang, W. (2016). Healthcare data gateways: Found healthcare intelligence on blockchain with novel privacy risk control. Journal of Medical Systems, 40(10), 218.

[23]

[24] international conference on Acoustics, speech and signal processing (ICASSP). IEEE, 2012,

[25] pp. 4277– 4280.

[26]

[27] D. Povey, "Discriminative training for large vocabulary speech recognition," Ph.D. dissertation, University of Cambridge, 2005.

[28] H. A. Bourlard and N. Morgan, Connectionist speech recognition: a hybrid approach. Springer Science & Business Media, 2012, vol. 247.

[29] R. M. Gray and D. L. Neuhoff, “Quantization,” IEEE transactions on information theory, vol. 44, no. 6, pp. 2325–2383, 1998.

[30] Y. Zhang, J. Qin, D. S. Park, W. Han, C.-C. Chiu, R. Pang, Q. V. Le, and Y. Wu, “Pushing the limits of semi-supervised learning for automatic speech recognition,” arXiv preprint arXiv:2010.10504, 2020.

[31] A. Graves, S. Fernandez, F. Gomez, and J. Schmidhuber, “Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks,” in ICML, 2006.

[32] A. van den Oord, O. Vinyals et al., “Neural discrete representation learning,” in NeurIPS, 2017.